

# Mocaverse Code Scanning — Response Report

Based on definitive code-level verification against the Mocaverse code scanning report.

- **Repository:** `mocachain/moca`
- **Branch:** `feat/e2e-validator-kind-migration`
- **Module path:** `github.com/evmos/evmos/v12`
- **Go version:** `go 1.23.6`
- **Audit date:** 2026-04-04

## 1. Critical CVEs

### 1.1 CVE-2024-32644 (CRITICAL) — Precompile stateDB.Commit()

| Item                | Detail                                                                                                                         |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Report claim</b> | Precompile calls <code>stateDB.Commit()</code> during execution, allowing partial state persistence and infinite token minting |
| <b>Verdict</b>      | <b>NOT VULNERABLE — attack path does not exist</b>                                                                             |

**Definitive evidence (3 layers of proof):**

- Interface isolation** — `go-ethereum/core/vm/interface.go:27-80` defines the `vm.StateDB` interface. This interface **does not include `Commit()`**. All precompiles access the state database via `evm.StateDB` which is typed as `vm.StateDB`. The Go compiler rejects any call to `Commit()` through this interface at compile time.
- No type assertion bypass** — Searched all 12 precompile packages (`x/evm/precompiles/*`) for type assertions like `.(*StateDB)` or `statedb.StateDB` — **zero matches**. No precompile code casts `vm.StateDB` to the concrete `*statedb.StateDB` type.
- No `Commit()` call in any precompile** — Searched the entire `x/evm/precompiles/` directory for any `.Commit()` call — **zero matches**. The only production `stateDB.Commit()` is in `x/evm/keeper/state_transition.go:416-421`, called **after** `evm.Call()/evm.Create()` has fully returned:

```
if commit {
    if err := stateDB.Commit(); err != nil {
        return nil, errorsmod.Wrap(err, "failed to commit stateDB")
    }
}
```

### 1.2 CVE-2024-37153 (CRITICAL) — ICS-20 Transfer Balance Not Deducted

| Item                | Detail                                                                                               |
|---------------------|------------------------------------------------------------------------------------------------------|
| <b>Report claim</b> | ICS-20 transfer from smart contract doesn't deduct contract balance, enabling infinite money via IBC |
| <b>Verdict</b>      | <b>NOT VULNERABLE — attack surface does not exist</b>                                                |

#### Definitive evidence (2 layers of proof):

- No ICS-20 transfer precompile exists** — The 12 registered precompiles in [app/app.go:1408-1474](#) are: bank, authz, gov, payment, permission, staking, distribution, storage, virtualgroup, storageprovider, slashing, erc20. **No `transfer/ibc/ics20` precompile is registered.** There is no precompile directory for IBC transfers under [x/evm/precompiles/](#).
- EOA-only restriction blocks indirect paths** — The `authz` precompile could theoretically execute `MsgTransfer` via its `Exec()` method. However, ALL write methods across ALL 11 precompiles with write operations enforce `evm.Origin == contract.Caller()`:

```
if evm.Origin != contract.Caller() {
    return nil, types.ErrInvalidCaller
}
```

When a smart contract calls a precompile, `contract.Caller()` is the contract address while `evm.Origin` is the EOA — they differ, so the call is **rejected**. Only EOA direct calls pass this check, and EOA calls deduct from the EOA's Cosmos bank balance correctly.

Verified in all 11 precompile [tx.go](#) files:

| Precompile      | EOA check count      | File                                                    |
|-----------------|----------------------|---------------------------------------------------------|
| bank            | 2                    | <a href="#">x/evm/precompiles/bank/tx.go</a>            |
| authz           | 3                    | <a href="#">x/evm/precompiles/authz/tx.go</a>           |
| gov             | 5                    | <a href="#">x/evm/precompiles/gov/tx.go</a>             |
| staking         | 5                    | <a href="#">x/evm/precompiles/staking/tx.go</a>         |
| distribution    | 7                    | <a href="#">x/evm/precompiles/distribution/tx.go</a>    |
| payment         | 4                    | <a href="#">x/evm/precompiles/payment/tx.go</a>         |
| storage         | 35                   | <a href="#">x/evm/precompiles/storage/tx.go</a>         |
| virtualgroup    | 10                   | <a href="#">x/evm/precompiles/virtualgroup/tx.go</a>    |
| storageprovider | 1                    | <a href="#">x/evm/precompiles/storageprovider/tx.go</a> |
| slashing        | 1                    | <a href="#">x/evm/precompiles/slashing/tx.go</a>        |
| erc20           | 2                    | <a href="#">x/evm/precompiles/erc20/tx.go</a>           |
| permission      | 0 (no write methods) | <a href="#">x/evm/precompiles/permission/tx.go</a>      |

### 1.3 GHSA-3fp5-2xwh-fxm6 (CRITICAL) — Non-atomic $A \rightarrow B \rightarrow A$ State Transitions

| Item         | Detail                                                                                                       |
|--------------|--------------------------------------------------------------------------------------------------------------|
| Report claim | Non-atomic $A \rightarrow B \rightarrow A$ state transitions bypass persistence, same class as infinite mint |
| Verdict      | <b>ARCHITECTURAL FLAW EXISTS, but NOT EXPLOITABLE due to EOA-only restriction</b>                            |

The vulnerability pattern exists at the architecture level (4 confirmed conditions):

1.  **$A \rightarrow B \rightarrow A$  skip logic confirmed** — `x/evm/statedb/statedb.go:507-509`, `Commit()` skips writing when `dirtyStorage[key] == originStorage[key]`:

```
for _, key := range obj.dirtyStorage.SortedKeys() {
    value := obj.dirtyStorage[key]
    // Skip noop changes, persist actual changes
    if value == obj.originStorage[key] {
        continue
    }
    s.keeper.SetState(s.ctx, obj.Address(), key, value.Bytes())
}
```

2. **Precompile `commit()` is immediate and irreversible** — All 12 precompiles follow the pattern: `CacheContext()` → execute → `commit()`. Once `commit()` executes, Cosmos state changes (e.g., bank balance transfers) are merged into the parent context and cannot be reversed.

```
ctx, commit := c.ctx.CacheContext()
snapshot := evm.StateDB.Snapshot()
```

```
commit()
return ret, nil
```

3. **EVM journal does NOT track Cosmos state** — `x/evm/statedb/journal.go` only tracks EVM-layer changes (balance, nonce, code, storage). `RevertToSnapshot()` cannot roll back precompile `commit()` changes.
4. **No atomic wrapping mechanism** — `RevertMultiStore`, `RunAtomic`, `WriteAccessList` — **zero matches** in the entire codebase.

**Why it is NOT exploitable today:**

The exploit requires a smart contract to call a precompile, then trigger a revert in the same call stack. But **ALL precompile write methods enforce `evm.Origin == contract.Caller()`**, blocking smart contract callers. When an EOA calls a precompile directly, there is no upper-layer contract that can revert after the precompile succeeds.

**Risk assessment:** The security depends entirely on a single check (`evm.Origin != contract.Caller()`). If any future precompile omits this check, the vulnerability becomes immediately exploitable. The root cause (non-atomic Cosmos/EVM state) remains unfixed.

### 1.4 GHSA-mjfq-3qr2-6g84 (HIGH) — Low Gas Partial Precompile Execution

| Item         | Detail                                                                                                  |
|--------------|---------------------------------------------------------------------------------------------------------|
| Report claim | Low gas limit causes partial precompile execution without revert, enabling fund theft or validator halt |
| Verdict      | ARCHITECTURAL FLAW EXISTS, but NOT EXPLOITABLE due to EOA-only restriction                              |

The vulnerability pattern exists (3 confirmed conditions):

- 1. **Missing fix symbols** — `RevertMultiStore`, `HandleGasError`, `RunAtomic` — **zero matches** in the entire codebase. The standard cosmos/evm fix has not been applied.
- 2. **RequiredGas() can return 0** — All 12 precompiles return 0 when method parsing fails or the method is unregistered. Example from `x/evm/precompiles/bank/contract.go:70-105`:

```
func (c *Contract) RequiredGas(input []byte) uint64 {
    method, err := GetMethodByID(input)
    if err != nil {
        return 0
    }
}
```

```
default:
    return 0
}
```

- 3. **EVM RevertToSnapshot cannot roll back Cosmos state** — Same root cause as GHSA-3fp5. If an outer EVM frame reverts after a precompile's `commit()` has executed, Cosmos state changes persist while EVM state is rolled back.

Attack scenario (theoretical):

```
Contract B calls Contract A
└─ Contract A calls precompile.Delegate(gas=60000)
    └─ RequiredGas() = 60000, UseGas(60000) ✓
        └─ Run() executes, commit() writes Cosmos staking state
            └─ Returns with gas=0
└─ Contract A OOG on next operation
└─ Contract B catches the revert: evm.StateDB.RevertToSnapshot()
└─ EVM state rolled back ✓, but Cosmos staking state persists ✗
```

**Why it is NOT exploitable today:** Same as GHSA-3fp5 — the EOA-only check (`evm.Origin != contract.Caller()`) prevents smart contracts from calling precompile write methods. Contract A cannot call `precompile.Delegate()` because `evm.Origin` (EOA)  $\neq$  `contract.Caller()` (Contract A).

---

## 2. Missing Security Features

### 2.1 Single-EVM-tx-per-Cosmos-tx Enforcement

| Item         | Detail                                                     |
|--------------|------------------------------------------------------------|
| Report claim | No restriction on EVM tx batching, allows batching attacks |
| Verdict      | CONFIRMED MISSING                                          |

`app/ante/evm/setup_ctx.go:56-58` explicitly handles "multiple eth msgs":

```
// Reset transient gas used to prepare the execution of current cosmos tx.
// Transient gas-used is necessary to sum the gas-used of cosmos tx,
when it contains multiple eth msgs.
esc.evmKeeper.ResetTransientGasUsed(ctx)
```

`EthEmitEventDecorator` at `setup_ctx.go:78-91` iterates over `tx.GetMsgs()` in a for-loop with no `len(msgs) == 1` validation.

---

### 2.2 Atomic Precompile Execution (RevertMultiStore)

| Item         | Detail                                                      |
|--------------|-------------------------------------------------------------|
| Report claim | No <code>RevertMultiStore()</code> , fundamental safety gap |
| Verdict      | CONFIRMED MISSING                                           |

Search for `RevertMultiStore` across the entire codebase — **zero matches**. Precompiles use `CacheContext` + EVM `Snapshot/RevertToSnapshot`, but these are independent systems. Cosmos `commit()` is irreversible once executed, while EVM `RevertToSnapshot` only rolls back journal entries.

---

### 2.3 IBC Rate-Limit Middleware

| Item         | Detail                                                          |
|--------------|-----------------------------------------------------------------|
| Report claim | No IBC rate limiting, no protection against rapid fund drainage |
| Verdict      | CONFIRMED MISSING                                               |

IBC transfer stack in `app/app.go:604-607`:

```
var transferStack porttypes.IBCModule
transferStack = transfer.NewIBCModule(app.TransferKeeper)
transferStack = erc20.NewIBCMiddleware(app.Erc20Keeper, transferStack)
```

Only `erc20` middleware is present. No `ratelimit` middleware. The `rate_limit` references in the codebase are exclusively storage module bucket flow control (`x/storage/`), unrelated to IBC.

---

## 2.4 Non-deterministic State Mutations in EVM Pre-blocker

| Item                | Detail                                                                    |
|---------------------|---------------------------------------------------------------------------|
| <b>Report claim</b> | Non-deterministic state mutations in EVM pre-blocker, consensus fork risk |
| <b>Verdict</b>      | <b>DOES NOT EXIST — no EVM PreBlocker registered</b>                      |

`app/app.go:778-780` — only `upgrade` module has a PreBlocker:

```
app.mm.SetOrderPreBlockers(
    upgradetypes.ModuleName,
)
```

EVM `BeginBlock` only calls `WithChainID(ctx)`. `EndBlock` only reads bloom from transient store and emits events (`x/evm/keeper/abci.go:25-42`). No non-deterministic sources.

---

## 3. Missing EVM Features

### 3.1 EIP-7702

| Item                | Detail                                               |
|---------------------|------------------------------------------------------|
| <b>Report claim</b> | Missing EIP-7702 (set EOA account code / delegation) |
| <b>Verdict</b>      | <b>CONFIRMED MISSING</b>                             |

Search for `7702` and `eip7702` across the entire codebase — **zero matches**. `SetCode` exists only in standard contract code storage (`StateDB/keeper`), not EIP-7702 delegation.

---

### 3.2 Custom EVM App-side Mempool

| Item                | Detail                              |
|---------------------|-------------------------------------|
| <b>Report claim</b> | Missing custom EVM app-side mempool |
| <b>Verdict</b>      | <b>CONFIRMED MISSING</b>            |

`app/app.go:358-364` uses `NoOpMempool` with default proposal handlers:

---

```

baseAppOptions = append(baseAppOptions, func(app *baseapp.BaseApp) {
    mempool := mempool.NoOpMempool{}
    app.SetMempool(mempool)
    handler := baseapp.NewDefaultProposalHandler(mempool, app)
    app.SetPrepareProposal(handler.PrepareProposalHandler())
    app.SetProcessProposal(handler.ProcessProposalHandler())
})

```

## 4. Summary

| #  | Issue                                      | Severity | Verdict                             | Detail                                                                                                                                                  |
|----|--------------------------------------------|----------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | CVE-2024-32644<br>(Precompile Commit)      | CRITICAL | <b>NOT VULNERABLE</b>               | <code>vm.StateDB</code> interface has no <code>Commit()</code> method; precompiles cannot call it. Compile-time guaranteed.                             |
| 2  | CVE-2024-37153<br>(ICS-20 balance)         | CRITICAL | <b>NOT VULNERABLE</b>               | No ICS-20 precompile exists; all write-capable precompiles enforce EOA-only access.                                                                     |
| 3  | GHSA-3fp5-2xwh-fxm6 (A → B → A non-atomic) | CRITICAL | <b>FLAW EXISTS, NOT EXPLOITABLE</b> | Architecture-level Cosmos/EVM non-atomicity confirmed. Currently blocked by EOA-only restriction on all precompile write methods. Single-layer defense. |
| 4  | GHSA-mjfq-3qr2-6g84 (low gas partial exec) | HIGH     | <b>FLAW EXISTS, NOT EXPLOITABLE</b> | Same root cause as #3. <code>RevertMultiStore/RunAtomic</code> not applied. Currently blocked by EOA-only restriction.                                  |
| 5  | Single-EVM-tx enforcement                  | Security | <b>MISSING</b>                      | AnteHandler processes multiple <code>MsgEthereumTx</code> per Cosmos tx without limit.                                                                  |
| 6  | Atomic precompile (RevertMultiStore)       | Security | <b>MISSING</b>                      | Zero matches for <code>RevertMultiStore</code> in codebase.                                                                                             |
| 7  | IBC rate-limit middleware                  | Security | <b>MISSING</b>                      | Transfer stack only has <code>erc20</code> middleware, no rate limiter.                                                                                 |
| 8  | EVM PreBlocker non-determinism             | Security | <b>DOES NOT EXIST</b>               | EVM has no PreBlocker. Begin/EndBlock are deterministic.                                                                                                |
| 9  | EIP-7702                                   | Feature  | <b>MISSING</b>                      | Zero matches for <code>7702/eip7702</code> .                                                                                                            |
| 10 | Custom EVM mempool                         | Feature  | <b>MISSING</b>                      | Uses <code>NoOpMempool</code> with default handlers.                                                                                                    |

## Risk Assessment

**Immediate risk: LOW** — The two architectural flaws (GHSA-3fp5, GHSA-mjfq) are real but currently blocked by the EOA-only restriction (`evm.Origin != contract.Caller()`) present in all 75 precompile write

method entry points across 11 precompile contracts.

**Latent risk: HIGH** — The EOA-only check is the **sole** defense against critical state inconsistency attacks. This is a fragile single-layer defense:

- Any new precompile that omits the EOA check → immediately exploitable
- Any refactoring that changes the check → immediately exploitable
- The root cause (Cosmos `commit()` not tracked by EVM journal) remains unfixed

## Priority Recommendations

### P0 (address promptly):

- Port `RevertMultiStore` / `RunAtomic` from Evmos v17+ or cosmos/evm to fix the Cosmos/EVM atomicity gap at the architecture level, eliminating reliance on EOA-only as the sole defense

### P1 (short-term):

- Add single-EVM-tx-per-Cosmos-tx enforcement in AnteHandler
- Add IBC rate-limit middleware to the transfer stack

### P2 (roadmap):

- EIP-7702 support
- Custom EVM app-side mempool